SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1.Introduction

SME Finance Ops Copilot is a strategic solution designed to transform how small and medium enterprises manage back-office finance operations. In many organizations, reimbursement claims, petty cash requests, vendor invoice approvals and internal payment processes are still handled through disconnected channels such as chat, email, spreadsheets and manual follow-ups. While this approach may work at low volume, it quickly becomes inefficient and risky as transaction complexity and business scale increase.

The core challenge is not only process fragmentation, but also the nature of financial evidence itself. Real operational inputs are often unstructured and inconsistent: blurry receipts, incomplete invoices, informal text descriptions, missing cost allocations, and mismatched amounts across documents. These conditions create repeated clarification loops, delayed approvals, duplicated submissions and inconsistent decision outcomes across departments. Over time, they also weaken traceability and make audit preparation significantly more difficult.

SME Finance Ops Copilot addresses this gap by introducing an intelligent, stateful workflow layer that converts messy financial inputs into structured, governable and auditable case flows. Instead of treating finance intake as a one-time form submission, the system treats each request as a living case that moves through controlled stages which are intake, extraction, clarification, policy evaluation, approval, finance review and export preparation. This creates operational continuity from submission to closure within a single system context.

A key design principle of the platform is governed AI assistance rather than unrestricted automation. AI supports extraction, reasoning and recommendation generation, but final financial actions remain controlled by explicit workflow rules and human decision points. When confidence is low or information is incomplete, the system does not force progression. It asks targeted follow-up questions and pauses safely until required context is provided. When risk conditions are detected, cases can be escalated to dedicated review lanes for controlled resolution.

This approach delivers both operational and governance value. Operationally, it reduces turnaround time, removes avoidable manual back-and-forth and improves queue efficiency for requesters, approvers and finance teams. From a control perspective, it strengthens policy adherence, preserves complete decision provenance and produces transparent audit trails across every transition, model output and human intervention. The result is a practical finance operations control plane that enables SMEs to digitize with discipline without the immediate burden of adopting a full-scale enterprise platform.

Overall, SME Finance Ops Copilot is positioned as a scalable foundation for modern SME finance execution: fast enough for day-to-day operations, structured enough for compliance and flexible enough to evolve into broader treasury and operational intelligence capabilities in future phases.

© UMHackathon 2026

1.1. Purpose

This system analysis document defines the technical purpose, implementation boundaries and core design decisions behind the development of SME Finance Ops Copilot. Its primary purpose is to explain how the product is engineered to convert unstructured finance requests into structured, auditable and policy-governed workflows for SME operations.

At a functional level, the system is built to support end-to-end handling of operational finance cases such as reimbursements, petty cash, invoice approvals and internal payment requests. The purpose is not only to digitize form submission, but to orchestrate the full lifecycle of each case which is from intake and evidence processing to review, approval, exception handling and export-ready closure within one consistent runtime flow.

From a technical perspective, this document clarifies the architecture and decision rationale used to make the system reliable under real-world finance conditions. The product is designed as a stateful workflow platform where each transition is explicit, traceable and governed by defined logic. AI is integrated as an assistive reasoning component for extraction, clarification and routing support, while deterministic validation, policy checks and human control points preserve operational safety and accountability.

A central purpose of this document is also to justify key design choices that balance automation with governance. These include structured output contracts, confidence-aware clarification loops, role-based approval lanes, recoverable exception states and complete action-level audit visibility. Together, these decisions ensure that the system can handle ambiguity without producing uncontrolled outcomes and can scale process efficiency without sacrificing compliance readiness.

This document further serves as a reference for alignment across product, engineering and evaluation stakeholders. It establishes what the current release is intended to deliver, how those capabilities are implemented, which constraints are intentionally accepted in this phase and how the platform is positioned for future expansion.

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. **Architecture**

SME Finance Ops Copilot follows a modular cloud-ready web platform architecture built for controlled finance workflow orchestration. The system is separated into clear layers so each part can evolve independently while still sharing one consistent operational model.

At the presentation layer, the web application serves multiple operational personas (requesters, approvers, finance reviewers, and admins) through role-specific workflow views. At the application layer, a backend API orchestrates case lifecycle transitions, policy evaluation, routing, exception handling and integration calls. At the shared contract layer, reusable domain schemas and workflow enums enforce consistent data structures between frontend and backend so transitions and validations are predictable. At the data layer, relational persistence stores case records, decisions, workflow states and audit events in a structured manner suitable for traceability and reporting.

AI capability is integrated through a governed gateway rather than directly embedded into user-facing logic. This allows extraction and reasoning outputs to be normalized, validated and bounded before any workflow transition is accepted. As a result, AI contributes intelligence but orchestration authority remains with explicit business logic and human checkpoints.

Overall, the architecture is intentionally designed for:

- Stateful workflow control rather than one-off task automation
- Auditability and governance across every decision stage
- Separation of concerns between UI, orchestration, shared contracts and persistence
- Scalable extensibility for additional finance modules and deeper integrations in future phases

© UMHackathon 2026

2. Data Flows

The system uses an end-to-end case-driven data flow that transforms unstructured finance evidence into controlled operational outcomes.

1. A user submits a finance case through the web interface, including notes and supporting artifacts (receipts, invoices, PDFs, images).
2. The backend validates payload shape and field constraints, then persists the initial case and evidence metadata.
3. Evidence text extraction and preprocessing generate machine-consumable context from uploaded inputs.
4. The AI reasoning layer receives bounded context and returns structured extraction plus routing intent.
5. Backend normalization and schema validation sanitize model output and reject malformed structures.
6. Policy evaluation and workflow rules determine the next valid lane (e.g., requester clarification, approval queue, finance review).
7. Human actors perform governed actions (approve, reject, request more info, assign, escalate), each recorded as timeline events.
8. When a case reaches completion criteria, export-ready structured outputs are generated for downstream accounting processes.
9. The user-facing system reflects updated status and decisions, and the case remains fully traceable from intake to closure.

This data flow ensures that every major transition is persisted, explainable, and recoverable, including failure paths such as incomplete evidence or export issues.

© UMHackathon 2026

3. Model Process

The model process in SME Finance Ops Copilot is designed as a controlled AI pipeline that converts unstructured finance evidence into structured, workflow-ready decisions while preserving reliability and governance. Instead of allowing direct free-form AI execution, the system uses staged processing with validation and human control checkpoints.

1. Case Intake and Evidence Submission
   A requester creates a case and submits notes with supporting artifacts (e.g., receipt images, invoice PDFs, or other files). This forms the raw evidence package for processing.
2. Preprocessing and Text Extraction Layer
   The system performs OCR and PDF text extraction to transform uploaded evidence into machine-readable text. This extracted content is combined with user-provided notes and case metadata to build the reasoning context.
3. Context Preparation and Input Bounding
   Before inference, the backend applies structural and size constraints to keep processing stable and predictable. The prepared context is then serialized into a strict machine-consumable payload.
4. Model Inference (Structured Reasoning Stage)
   The AI model receives the prepared context and produces structured output focused on:
   - extracted finance-relevant fields,
   - confidence and evidence quality indicators,
   - clarification questions for missing/ambiguous data,
   - recommended routing or transition intent.
5. Post-Inference Normalization and Validation
   Model output is normalized into canonical system fields and validated against shared schemas and workflow rules. Malformed or unusable outputs are not trusted as final decisions.
6. Policy and Workflow Decision Gate
   The validated output is evaluated against policy logic (such as completeness checks, risk signals, and routing thresholds).
   - If sufficient and compliant, the case progresses to approval or finance review lanes.
   - If incomplete or uncertain, the system routes the case into clarification, requesting additional input from the requester.
7. Human-in-the-Loop Control
   Approvers and finance reviewers remain the final decision authorities at governance checkpoints. AI outputs act as decision support, not autonomous finalization.
8. Lifecycle Continuation and Audit Logging
   Every transition, AI result, clarification, and human action is recorded in the case timeline. The same case iterates through this loop until it reaches a valid completion and export-ready state.

© UMHackathon 2026

4. Role of Reference

This document serves as a shared technical reference for all project stakeholders involved in design, implementation, validation, and maintenance.

Its role is to:

- Align developers on architectural boundaries, workflow contracts, and orchestration responsibilities
- Align testers on expected lifecycle behavior, transition logic, exception paths, and verification checkpoints
- Provide product and review stakeholders with a clear explanation of why major technical decisions were made
- Establish a stable baseline for future enhancements, refactoring, and scope expansion
- Reduce ambiguity during collaboration by standardizing terminology for states, lanes, controls, and outputs

© UMHackathon 2026

Email: umhackathon@um.edu.my

1.2. Background

Across finance operations, many daily processes still depend on fragmented tools, manual review steps, and unstructured evidence such as notes, receipts, invoices, screenshots, and email style requests. Teams often copy information between systems, interpret incomplete data by hand, and repeat the same routing decisions for each case. This creates long turnaround times, inconsistent decision quality, poor traceability, and frequent rework when information is missing or ambiguous.

The system was designed as an AI powered workflow platform where Z AI GLM serves as the central reasoning engine. The objective is not generic text generation. The objective is a stateful workflow engine that can understand unstructured inputs, perform multi step decisions, orchestrate downstream actions, and return structured outputs that drive operational state transitions. Without the GLM reasoning layer, the system loses core capability to interpret evidence quality, generate clarification posture, and coordinate routing decisions effectively.

Before this implementation, the operating model was closer to manual coordination than integrated intelligent workflow execution. Case intake, evidence interpretation, approval handling, and exception management behaved like loosely connected activities with high human mediation. Decision rationale and process history were harder to consolidate in one place, and routing consistency depended heavily on individual operator judgment.

The new architecture introduces coordinated web, API, and shared contract layers with explicit states and role based lanes. The frontend now provides dedicated requester, approver, finance reviewer, dashboard, case detail, and admin control surfaces. The backend now orchestrates intake, policy evaluation, approval transitions, finance review actions, export handling, and recovery flows. Shared schemas and contracts now keep request and response structures consistent across client and server to reduce integration drift.

An AI gateway layer now links orchestration with GLM reasoning outputs including extraction fields, confidence, provenance, clarification questions, and next state intent. Document services normalize extraction from text, PDF, and image evidence before reasoning, and audit plus transition persistence improves traceability across every workflow stage and operator action.

The resulting version introduces intelligent evidence to decision flow where unstructured input becomes actionable workflow data, clarification loops that safely pause and resume incomplete cases, dual control queues for approval and finance resolution, recoverable exception handling for controlled failure recovery, and unified operational visibility through dashboard and case timeline surfaces. Together these changes move operations from fragmented manual coordination toward adaptive stateful workflow execution.

© UMHackathon 2026

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
Requester users	Create finance cases, upload evidence, provide business notes, and answer clarification requests	Fast submission flow, clear status visibility, simple clarification steps, and confidence that cases progress without repeated manual follow up
Approver users	Review approval queue items and take approve, reject, request info, or delegate actions	Complete case context for decisions, low friction action controls, traceable rationale capture, and reliable queue accuracy after each action
Finance reviewer users	Resolve escalated cases through finance review decisions, assignment, reconciliation metadata, and send back flows	Strong exception handling controls, structured review fields, clear ownership visibility, and governed transitions before export
Admin users	Configure policy thresholds, routing defaults, delegations, and matrix settings while monitoring connector readiness	Runtime control over governance parameters, safe configuration updates, and visibility into operational readiness signals
Operations lead or queue manager	Monitor dashboard workload, lane pressure, backlog signals, and case distribution across states	Actionable operational visibility, early bottleneck detection, and fast navigation from monitoring surfaces to intervention points
Compliance and audit stakeholders	Inspect transition history, decision records, and exception handling paths for each case	End to end traceability, transparent decision lineage, and auditable evidence of control enforcement across workflow stages
Platform engineering team	Build, maintain, and evolve web, API, queue, and data layers of the workflow platform	Modular architecture, stable contracts, maintainable code boundaries, predictable runtime behavior, and clear extension points for future capabilities

© UMHackathon 2026

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview

Type	Details
System	Web-based multi-role platform (Requester, Approver, Finance Reviewer, Admin) with backend API services
Architecture	Cloud-ready client-server architecture with modular service layers (Web App + API + Shared Contracts + Data/Queue/AI integrations)

SME Finance Ops Copilot is structured as a client-server finance workflow platform. The user-facing client is a web application that provides role-specific interfaces for case submission, approval actions, exception review, and policy administration. All client actions communicate through a centralized backend API layer that controls workflow state transitions, policy enforcement, AI-assisted reasoning orchestration, and audit logging.

At a high level, the architecture consists of:

- Frontend Layer (apps/web)
  Next.js web application for operational users. It provides lane-based workflow views and role-specific actions across the case lifecycle.

- Backend Orchestration Layer (apps/api)
  NestJS API that acts as the workflow control plane. It handles case intake, validation, OCR/PDF evidence processing coordination, AI gateway calls, routing decisions, queue handling, approval/finance-review operations, and export flow.

- Shared Contract Layer (packages/shared)
  Common domain models, schemas, and workflow enums used by both frontend and backend to ensure consistent data shapes and deterministic transition behavior.

- Persistence and Runtime Services
  PostgreSQL (via Prisma) stores cases, decisions, workflow states, and audit events. Redis/BullMQ supports queue-oriented processing and asynchronous workload handling where needed.

- External Integration Layer
  AI provider gateway (Z.AI GLM), storage/auth connectors, and downstream export/integration targets are accessed through controlled backend interfaces, not directly from client UI.

© UMHackathon 2026

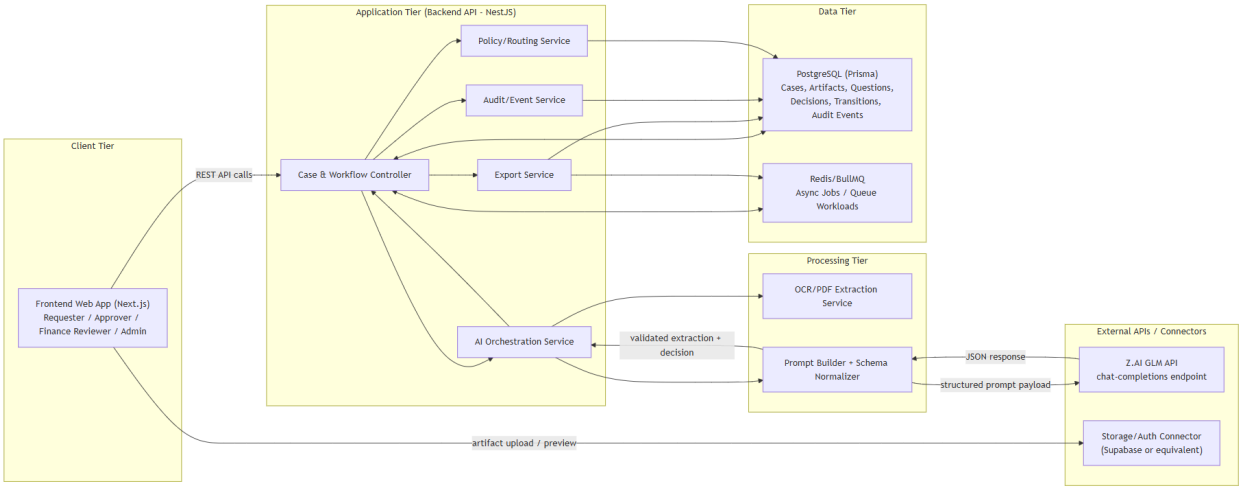
This architecture is intentionally designed so that AI remains assistive, while workflow logic, policy rules, and human checkpoints remain authoritative. As a result, the platform can automate high-friction finance operations while preserving traceability, governance, and safe operational control.

2.1.2 LLM as Service Layer

The architecture of your product needs to show how GLM is integrated as a service layer- not limited to a generic “AI Module”. The following Dependency diagram includes:

2.1.3 Dependency Diagram

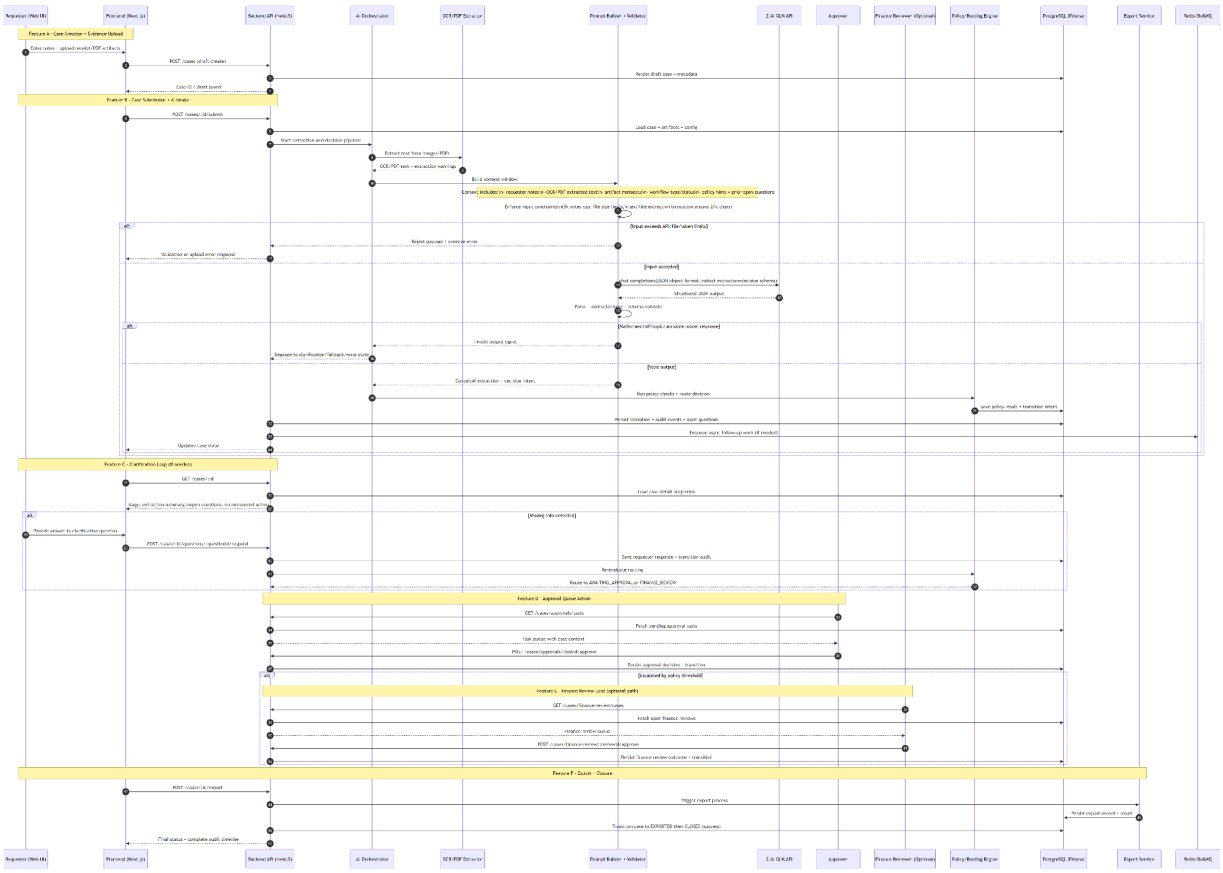
1) Distributed Component / Dependency Diagram



- **Distributed deployment intent:** client tier, API tier, processing tier, data tier, and external API tier can scale independently.
- **Control principle:** AI is assistive; workflow transitions and approvals stay enforced by backend policy/state logic.

© UMHackathon 2026

2.2.2. Sequence Diagram



© UMHackathon 2026

Email: umhackathon@um.edu.my

2.2. Technological Stack

2.2.1 Frontend

Frontend uses Next.js with React and App Router patterns for role based operational pages. The implementation provides requester submission flow, case detail operations, approval queue interactions, finance review handling, dashboard visibility, and admin configuration surfaces. Shared contract schemas are consumed in the frontend to validate and parse API responses consistently.

2.2.2 Backend

Backend uses NestJS for API and workflow orchestration services. The backend provides intake orchestration, AI gateway integration with Z AI GLM, policy evaluation, approval and finance review transitions, export processing, artifact processing, audit logging, and recovery flow control. Queue execution supports inline mode and worker backed mode through BullMQ integration patterns.

2.2.3 Database

Primary persistence uses PostgreSQL with Prisma as ORM and schema management layer. The relational model stores core entities such as cases, artifacts, extraction results, open questions, policy results, approval tasks, finance reviews, export records, transitions, and audit events. This structure supports stateful workflow progression with traceable historical context.

2.2.4 Cache and queue infrastructure

Redis is used as queue infrastructure for asynchronous job execution when worker mode is enabled. Queue categories include artifact processing, AI intake, policy evaluation, and export processing. The design also supports a local inline fallback execution path for controlled development operation.

2.2.5 Storage and document handling

The current implementation supports local artifact storage for uploaded evidence and includes extraction services for text, PDF, and image inputs. OCR is handled through Tesseract based processing for image artifacts, while PDF text extraction is handled through dedicated parsing services before reasoning and routing.

2.2.6 Authentication and authorization

Authentication and role handling are implemented through application sessions and role aware backend guards. Authorization checks enforce lane specific access for requester, approver, finance reviewer, and admin operations across case, queue, and configuration endpoints.

© UMHackathon 2026

2.2.7 Shared contracts and schemas

The shared package provides cross application domain types, workflow enums, Zod schemas, and API contracts. This layer keeps client and server interfaces aligned and reduces runtime mismatch risk for critical response shapes such as case detail, approval queue, finance review queue, and admin settings.

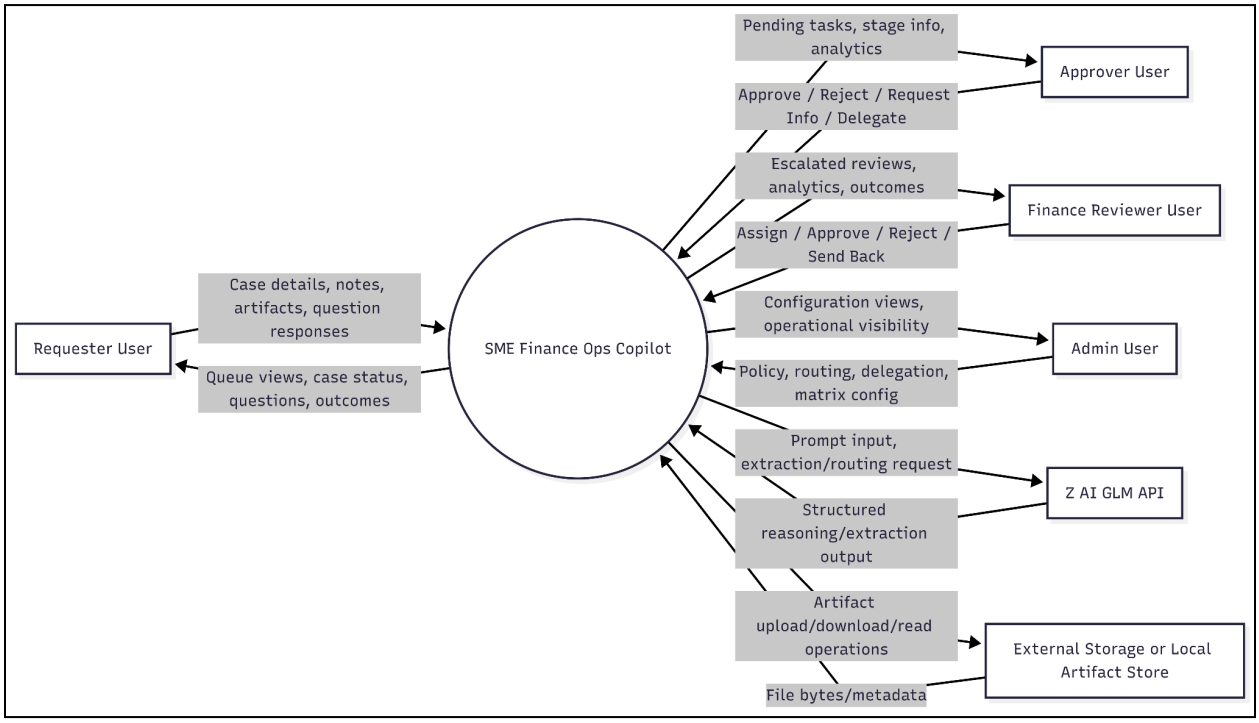
2.2.8 Cloud and deployment posture

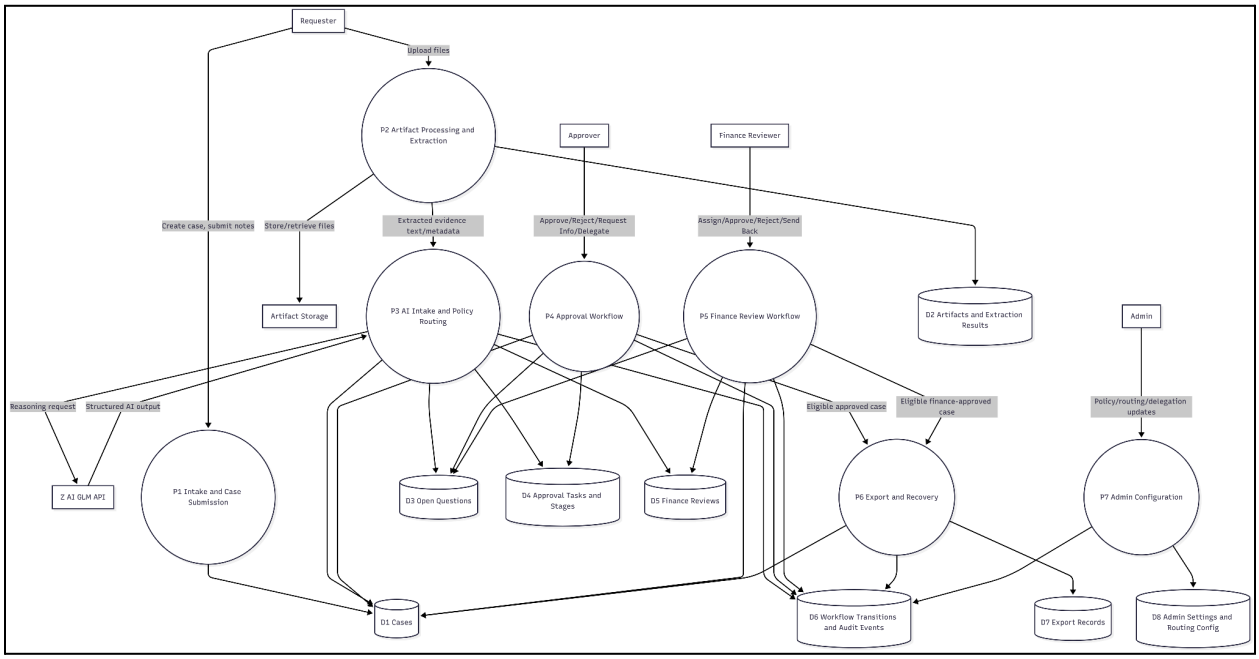
The current project setup is optimized for local and controlled environment execution with Docker Compose support for PostgreSQL and Redis. Runtime configuration also includes integration ready paths for external storage and auth providers where environment settings are available. Production cloud topology can be deployed on standard container or virtual machine infrastructure with managed PostgreSQL, Redis, and object storage equivalents according to organizational deployment policy.

© UMHackathon 2026

2.3. Key Data Flows

2.3.1. Data Flow Diagram (DFD)

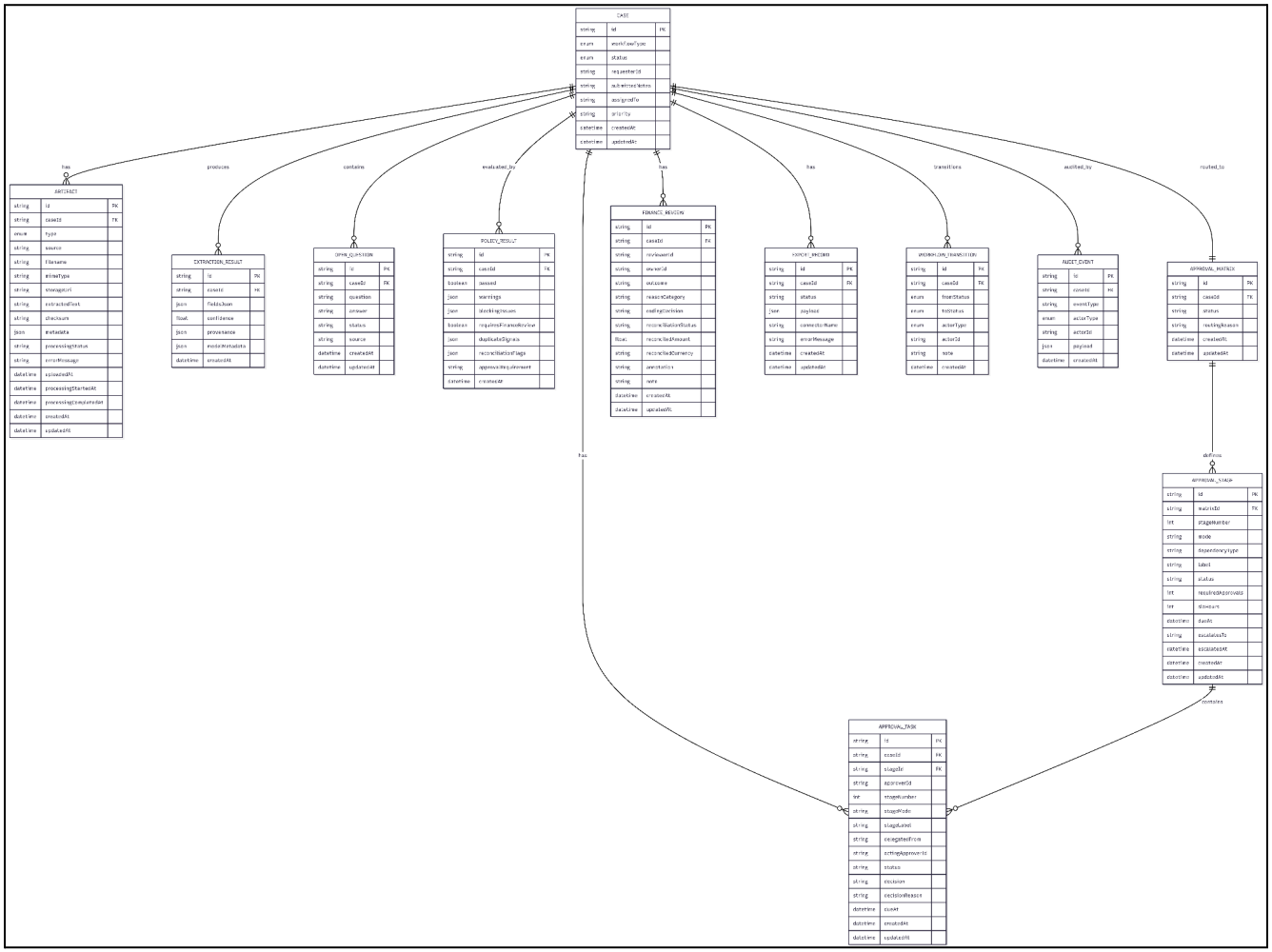




© UMHackathon 2026

Email: umhackathon@um.edu.my

2.3.2. Normalized Database Schema



© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Functional Requirements & Scope

3.1. Minimum Viable Product

#	Features	Description
1	Case Intake & Evidence Submission	Requester creates a finance case, selects workflow type, uploads supporting artifacts (receipt/invoice/PDF/image), and submits notes to start processing.
2	AI-Assisted Intake & Policy Routing	The system extracts structured context from evidence, evaluates policy conditions, and routes each case to the correct lane (approval, finance review, or clarification).
3	Approval Workflow Actions	Approver queue supports approve, reject, request-info, and delegate actions with stage-aware progression and decision traceability.
4	Finance Review Workflow Actions	Finance review queue supports assign owner, approve, reject, and send-back actions with structured review metadata and requester follow-up loop support.
5	Export & Recovery with Audit Trail	Cases progress to export-ready/export flow, failed export paths move to recoverable exception, and all major actions are recorded via transitions and audit events.

© UMHackathon 2026

3.2. Non-Functional Requirements (NFRs)

Reliability	Workflow transitions must stay valid under concurrent updates so stale state writes do not corrupt case progression.	Backend transition guards enforce expected from state checks and reject invalid transition attempts.
Availability	Core submission, queue, and case detail flows should remain usable during infrastructure mode changes.	Queue execution supports worker backed processing plus inline fallback execution for runtime continuity.
Fault tolerance	Failures in AI intake or export should not cause silent data loss or irreversible workflow breakage	Recoverable exception state, audit events, and manual recovery actions provide controlled remediation flow
Input robustness	Unstructured and low quality inputs should be handled safely without unsupported field invention	Prompt constraints, extraction warnings, clarification loops, and schema validation gate unsafe progression
Data integrity	Evidence and workflow history must be traceable for each case from intake to closure	Case detail includes transitions, audit events, extraction records, policy results, and decision history
Security and access control	Only authorized roles should perform lane specific actions and sensitive transitions	Role aware guards and authorization checks enforce requester, approver, finance reviewer, and admin boundaries
Token latency	AI assisted intake response time should remain stable for normal evidence volume and avoid uncontrolled delay.	Single constrained AI call pattern with bounded note and extraction inputs limits unnecessary token expansion.
Cost efficiency	Average inference cost per active submission session should stay within an operational budget envelope.	Bounded input design, strict structured output format, and no default multi pass reasoning reduce token spend.
Scalability	The system should support growth in workflow volume and asynchronous task throughput	Redis plus queue based job categories and modular orchestration services provide scale ready separation of concerns

© UMHackathon 2026

Maintainability	Frontend and backend interfaces should remain consistent to reduce integration drift	Shared contracts and Zod schemas enforce aligned request and response shapes across layers
Observability	Operational teams must be able to detect bottlenecks and inspect system behavior quickly	Dashboard metrics, queue analytics, connector status, and audit timeline surfaces provide runtime visibility

3.3. Out of Scope / Future Enhancements

Future Enhancements:

- Enterprise-grade approval matrix depth
  Current routing supports core approval lanes, but full multi-tier, condition-heavy approval matrix logic across all organizational structures is not included yet.

- Advanced foreign exchange (FX) reconciliation
  Multi-currency reconciliation logic, settlement-level controls, and deeper FX exception automation are reserved for future phases.

- Comprehensive accounting-grade validation engine
  Full consistency validation for net/tax/gross across all document types and workflow permutations is not fully implemented in this release.

- Advanced duplicate and fraud reasoning
  Baseline duplicate detection exists, but richer anomaly intelligence (cross-case similarity, behavioral fraud signals, forensic checks) is not part of the current scope.

- Dedicated oversized-context chunk-and-merge AI pipeline
  The present system relies on bounded input limits and deterministic handling; full multi-pass chunking/merging for very large evidence contexts is not implemented.

- AI retry orchestration with backoff policies
  Sophisticated retry budgets, adaptive retry strategy, and model-routing-by-failure-class are not yet included.

- Specialized hallucination adjudication module
  There is no standalone secondary verifier/arbiter model layer yet; current safeguards rely on schema constraints, prompt discipline, and human checkpoints.

- True real-time co-editing of the same case form

© UMHackathon 2026

Multi-user workflow participation is supported, but live concurrent editing with shared cursors is outside this release.

- Native mobile applications (iOS/Android)
  Delivery is web-first for this phase; dedicated native mobile clients are not included.

- Broad workflow family expansion beyond current finance lanes
  Scope is currently centered on core SME finance operations, not full enterprise process coverage.

- Autonomous finance control tower
  Evolve from workflow assistance to proactive operations intelligence: predictive bottleneck alerts, SLA breach forecasting, and suggested corrective actions.

- Treasury and liquidity intelligence modules
  Extend beyond case handling into cash forecasting, payment timing optimization, and working-capital decision support.

- Adaptive model orchestration layer
  Introduce tiered model routing, selective caching, and cost-aware inference policies to optimize latency, quality, and token spend at scale.

- Risk and compliance intelligence
  Add stronger confidence gating, anomaly scoring, explainable risk labels, and policy simulation tools for finance governance teams.

- Deep ERP/accounting ecosystem integrations
  Move from export-ready outputs toward richer two-way integration patterns with accounting/ERP stacks and reconciliation systems.

- Workload balancing and reviewer optimization
  Implement automated assignment intelligence to reduce reviewer bottlenecks and improve throughput across approval and finance-review lanes.

- Mobile and omnichannel submission experience
  Expand intake channels (native mobile, chat-assisted intake, assisted capture) while preserving centralized governance and auditability.

© UMHackathon 2026

Email: umhackathon@um.edu.my

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

The team evaluates SME Finance Ops Copilot using a production-like operational quality model rather than only UI correctness. Evaluation is measured across five dimensions:

- Workflow correctness: valid state transitions, no illegal state drift, and deterministic routing behavior.
- AI output reliability: structured response validity, extraction completeness, clarification quality, and fallback behavior under weak evidence.
- Operational throughput: case cycle time, queue aging, reviewer load, clarification loop frequency, and export success rate.
- Governance quality: audit trail completeness, policy enforcement consistency, and human decision traceability.
- Resilience: behavior under connector degradation, model response failures, and recoverable exception flows.

Core success indicators include:

- reduced average time from submission to closure,
- lower manual back-and-forth per case,
- high structured extraction acceptance before human correction,
- stable approval/finance-review queue turnaround,
- low unresolved exception backlog.

© UMHackathon 2026

4.1.1. Grayscale Rollout & A/B Testing

Grayscale Rollout

New high-impact capabilities (especially AI/policy/routing logic) are released incrementally by traffic segment, role group, or workflow type.

Suggested progression:

- Stage 0 - Internal/Shadow: internal team only, with full observability and manual review.
- Stage 1 - Limited cohort (5%): low-risk users/cases, strict alert thresholds.
- Stage 2 - Expanded cohort (20-30%): broader role coverage with active queue monitoring.
- Stage 3 - Majority rollout (50-75%): scaled load with exception stress checks.
- Stage 4 - Full rollout (100%): only after stability, quality, and governance KPIs pass.

Rollout gates must pass before promotion:

- no critical transition integrity regressions,
- acceptable AI structured-output validity rate,
- no abnormal spike in clarification loops or reviewer backlog,
- export failure and recoverable exception rates within limits.

Rollback/containment controls:

- feature flags for AI routing paths,
- controlled fallback modes,
- immediate cohort rollback if SLA or correctness thresholds are breached.

A/B Testing

After feature stability is confirmed, A/B testing is used to optimize business outcomes.

Typical experiments:

- Clarification prompt strategy: concise vs guided multi-question prompts.
- Queue prioritization strategy: amount-first vs risk-first sorting.
- Decision-support presentation: compact rationale vs evidence-first rationale cards.
- Policy threshold messaging: strict warning vs contextual coaching.

Primary A/B metrics:

- clarification completion rate,
- time-to-approval,
- approver decision confidence (proxy through rework/request-info frequency),
- finance review escalation precision,
- case closure SLA attainment.

Guardrails during A/B:

- no test may bypass governance controls,
- all variants must preserve auditability and explicit human checkpoints,
- high-risk finance actions stay policy-guarded regardless of variant.

© UMHackathon 2026

4.1.2. **Emergency Rollback (ER) & Golden Release**:

Golden Release Baseline

A Golden Release is the latest verified stable build that has passed technical and operational gates (workflow correctness, queue stability, export reliability, and AI response safety checks).

This release is maintained as the immediate recovery target and is always deployable.
Golden Release criteria:

- all critical workflow transitions validated,
- no high-severity regression in approval/finance-review flows,
- acceptable API error and timeout rates,
- stable AI parsing/validation behavior,
- export path and recoverable exception handling verified.

Emergency Rollback (ER) Trigger Conditions

ER is triggered when production-like monitoring detects critical anomalies, such as:

- invalid or drifting workflow transitions,
- sustained API error spike or severe latency degradation,
- abnormal AI malformed-response or parse-failure surge,
- queue backlog growth beyond defined SLA thresholds,
- export failure spikes causing operational blockage,
- policy-routing misbehavior affecting decision governance.

ER can be initiated manually by on-call owners or automatically through pipeline guardrails when hard thresholds are breached.

ER Recovery Procedure

When ER is activated, the system executes controlled recovery steps:

1. Freeze progressive rollout and halt further traffic expansion.
2. Disable high-risk feature flags (new AI routing/policy paths) to reduce blast radius.
3. Shift traffic to Golden Release through deployment rollback controls.
4. Preserve in-flight case data in persistent storage; no destructive rollback of business records.
5. Activate degraded-safe mode if needed (clarification-first/manual-review-first posture).
6. Validate health checks (API, queues, DB, export path, transition consistency).
7. Resume traffic gradually only after post-rollback verification passes.
8. This ensures continuity while keeping human governance and auditability intact during incident recovery.

Traffic Management During Incidents

Traffic is managed in controlled stages to avoid repeated failure:

- reduce exposure to low-risk cohorts first,
- prioritize essential workflow APIs over non-critical operations,
- throttle non-urgent background workloads if queue pressure is high,

© UMHackathon 2026

- keep approval and finance-review decision routes available,
- isolate unstable connectors behind fallback controls.

This minimizes business disruption while preserving critical case progression.

Post-Incident Controls

After ER recovery, the team performs:

- root-cause analysis and corrective patching,
- replay/verification on affected case paths,
- regression checks against transition integrity and export outcomes,
- controlled re-entry rollout (5% -> 20% -> 50% -> 100%),
- Golden Release refresh only after stability reconfirmation.

© UMHackathon 2026

4.1.3. Priority Matrix

P1 - Critical (Immediate Business/Governance Risk)

Typical triggers

- Invalid workflow transitions that can cause wrong case state progression.
- Approval/finance decision actions failing broadly (sustained endpoint failure).
- Export failure rate spikes and blocks financial closure operations.
- Severe API outage/latency causing system-wide operational disruption.
- AI response/parsing failure surge that prevents reliable case intake routing.
- Data integrity anomalies affecting case/audit consistency.

Action

- Trigger Emergency Rollback (ER) immediately.
- Route traffic back to Golden Release.
- Disable high-risk feature flags (new AI/policy/routing paths).
- Freeze non-essential deployments and traffic expansion.
- Activate incident bridge + on-call owners (API/workflow/data).
- Preserve in-flight records, run integrity checks, and publish incident updates.
- Resume rollout only after hotfix + verification gates pass.

P2 - High (Major Degradation, Workaround Available)

Typical triggers

- Approval queue aging or finance-review backlog exceeds SLA thresholds.
- Clarification loop spikes indicating low extraction quality.
- Increased fallback/degraded AI mode usage with rising manual load.
- Partial connector instability (storage/auth/integration delays).
- Elevated but non-catastrophic export retry/failure rates.

Action

- Keep system online, but enter stabilization mode.
- Pause rollout progression; hold at current cohort.
- Shift routing toward safer/manual-review-first behavior.
- Apply targeted fixes/config tuning (thresholds, queue policy, rate controls).
- Increase monitoring cadence and owner check-ins.
- Promote to P1 if thresholds worsen or governance risk appears.

P3 - Medium (Localized Issue / Limited Impact)

Typical triggers

- Non-critical endpoint regressions with available alternatives.
- Minor workflow UX issues not blocking case lifecycle completion.
- Temporary latency increase in non-core views.
- Intermittent queue lag with no SLA breach yet.

Action

- Log incident ticket and assign owner.

© UMHackathon 2026

- Apply patch in next release window.
- Monitor trend to ensure no escalation.
- Add regression test coverage for affected path.

P4 - Low (Cosmetic / Improvement)

Typical triggers

- UI text/formatting inconsistencies.
- Minor dashboard metric display mismatch.
- Documentation/reporting discrepancies without runtime impact.

Action

- Add to backlog for normal sprint planning.
- Bundle with routine maintenance release.
- No rollback or traffic control needed.

© UMHackathon 2026

Email: umhackathon@um.edu.my

4.2. Monitoring

4.2.1. Monitoring Plan

The monitoring model combines system telemetry and workflow intelligence:

- Application/API health: latency, error rates, timeout rates, connector health.
- AI pipeline health: parse/validation failure rate, fallback frequency, malformed response incidents.
- Workflow health: stage distribution, transition anomalies, stuck-case detection, reopen loops.
- Queue operations: pending count, age buckets, per-role workload imbalance.
- Export integrity: success/failure trends, recovery completion rates.
- Governance auditability: completeness of timeline events and decision provenance.

Alerting examples:

- sudden increase in AWAITING_REQUESTER_INFO,
- sustained growth in RECOVERABLE_EXCEPTION,
- spike in invalid model payload parsing failures,
- approval queue aging beyond SLA thresholds.

4.3. Assumptions

The following operational and environmental assumptions are considered valid during development, testing, and deployment of SME Finance Ops Copilot:

- Users (requesters, approvers, finance reviewers, admins) have stable internet access when submitting cases, reviewing tasks, and completing decisions.
- Organizations have access to desktop/laptop devices for web-based usage of all workflow lanes.
- Uploaded evidence files (receipts, invoices, PDFs, images) are generally readable and within configured size/type limits.
- Users provide minimally sufficient context in notes and clarification responses for cases to progress.
- Role ownership is correctly provisioned so each actor can access only their permitted workflow actions.
- The backend API environment, database service, and queue runtime are available with acceptable uptime for daily operations.
- External AI service and connector dependencies are reachable during active processing windows.
- Policy and routing configurations are maintained by admin users and updated when organizational rules change.
- Human checkpoints remain available (approver/finance reviewer capacity exists) to prevent long queue stagnation.

© UMHackathon 2026

- Time synchronization and environment configuration are consistent enough to preserve ordered audit timelines.
- Secure handling of credentials, environment variables, and integration keys is managed by the deployment environment.
- Monitoring and alerting channels are active so incidents can be detected and handled within expected response windows.

© UMHackathon 2026

Email: umhackathon@um.edu.my

4.4. *External Dependencies*

Tools	Purpose	Risks
Z AI GLM API (ZAI_BASE_URL, ZAI_MODEL_PRIMARY , ZAI_MODEL_FAST)	Core LLM reasoning for intake analysis, structured extraction intent, and workflow routing recommendations.	High: if API is unavailable, rate-limited, or slow, AI-assisted flow degrades. Mitigation: fallback behavior, bounded inputs, and manual clarification/review lanes.
PostgreSQL	Primary transactional database for cases, artifacts, approvals, finance reviews, transitions, and audit events.	High: schema drift or DB downtime blocks core business operations. Mitigation: Prisma schema sync, backup strategy, migration/runbook discipline.
Redis	Queue and async infrastructure (used by BullMQ when not running fully inline).	Medium: queue instability can delay background processing (artifact/AI/policy/export jobs). Mitigation: inline fallback mode, queue health checks, retry controls.
Prisma ORM	Data access layer and schema management between NestJS services and PostgreSQL.	Medium-High: client/schema mismatch causes runtime failures (e.g., missing columns). Mitigation: run prisma db push/migrations + prisma generate during updates.
Supabase Auth/JWT	Token verification and role-based identity integration when not using mock auth.	Medium: invalid or missing JWT secret/keys breaks authenticated routes. Mitigation: environment validation and role/session smoke tests.
Supabase Storage Bucket	External object storage integration path for artifact files (when enabled).	Medium: storage auth/config issues can break upload/download path. Mitigation: local storage fallback and connector health checks.
Tesseract OCR	OCR extraction for image artifacts during document processing.	Medium: OCR quality/performance varies with image quality and file size. Mitigation: warnings, manual review loop, bounded processing and retries.

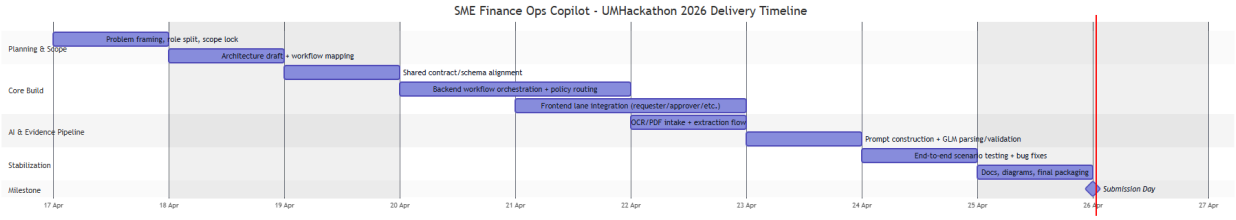
© UMHackathon 2026

PDF.js	Text extraction from PDF documents for downstream AI/policy processing.	Low-Medium: scanned/no-text PDFs may return empty extraction. Mitigation: extraction warning handling and fallback to clarification/manual review.
Docker Compose	Local reproducible runtime environment for DB and queue dependencies.	Low-Medium: container not running/misconfigured ports causes API startup/runtime errors. Mitigation: standard docker compose up -d and reset runbook.

© UMHackathon 2026

5. Project Management & Team Contributions

5.1. Project Timeline:



5.2. Team Members Role

First and foremost, Low Kah Xuan serves as backend lead and owns the API and workflow orchestration layer. This role covers backend module design, endpoint behavior, state transition reliability, and queue execution pathways across approvals, finance review, export, and recovery flows. He is also responsible for backend engineering documentation, including service architecture notes, endpoint behavior references, and backend change summaries for each milestone.

Next, Lui Qiu Yang serves as data and schema lead and owns Prisma schema evolution, relational model consistency, and database synchronization across environments. This role ensures schema changes remain aligned with service assumptions and shared contracts while preserving migration safety and data integrity. His effort in documentation includes schema definitions, entity relationship explanations, migration and reset runbooks, and data dictionary updates for major fields used in workflow decisions.

On the other hand, Ong Qi Tzern serves as Frontend Lead and owns user-facing workflow implementation in the web application. This includes requester submission flow, approvals page interactions, finance review operations, dashboard visibility, and role-aware UI behavior. He also owns frontend documentation such as page flow descriptions, component behavior notes, and operator usage walkthroughs for common actions and error recovery paths.

Furthermore, Lee Peng Jin serves as QA and integration lead and owns cross-lane test validation and release verification. This role defines test scenarios across requester, approver, finance reviewer, and admin paths, verifies transition correctness under edge cases, and confirms fixes through regression checks before release. He documented test plans, defect logs, validation evidence, retest outcomes, and quality sign-off records per sprint cycle.

© UMHackathon 2026

Lastly, Lui Qi Theng serves as UI/UX and product design lead and owns interaction quality, usability consistency, and design direction across requester, approver, finance review, and admin experiences. This role defines user flows, information hierarchy, visual consistency, and feedback behavior so that workflow actions are clear, efficient, and easy to verify. His documentation effort includes user journey maps, interface rationale, design decisions, usability findings, and visual guideline notes that support implementation consistency and strengthen final project communication.

© UMHackathon 2026

5.3. Recommendations:

Scalability

- Introduce workload-based queue partitioning
  Split async jobs by workflow type and priority (e.g., extraction, policy, export) to avoid one heavy lane blocking others.

- Adopt horizontal API scaling with stateless service design
  Keep orchestration stateless at runtime and rely on shared persistence/queue systems so additional API instances can be added under load.

- Implement adaptive model routing
  Route low-complexity cases to lighter AI paths and reserve higher-cost reasoning for ambiguous/high-risk cases to sustain growth efficiently.

- Add tenant/organization-aware throttling
  Prevent one tenant's burst traffic from degrading platform-wide performance.

- Plan read-model optimization for dashboard and queue views
  Use denormalized query projections/materialized views for high-frequency operational screens.

Performance

- Add selective caching for expensive reads
  Cache stable admin config, policy/routing metadata, and dashboard aggregates (Redis) with short TTL and explicit invalidation.

- Optimize case-detail query paths
  Reduce heavy nested retrieval by supporting "summary-first + lazy detail fetch" patterns for large cases.

- Use async extraction and export pipelines by default
  Move long-running OCR/PDF and export operations to queues with progress status updates to keep API response times stable.

- Apply payload minimization before AI calls
  Continue strict context bounding; add smarter evidence ranking/summarization before model invocation to reduce token and latency cost.

- Enable connection pooling and DB index tuning
  Index transition-heavy and queue-filtered columns (status, assignee, createdAt, updatedAt, priority) to stabilize throughput.

© UMHackathon 2026

Reliability

- Formalize circuit breakers and retry budgets per external dependency
  Define bounded retries, timeout policies, and fallback behavior for AI/storage/connectors to prevent cascade failures.

- Strengthen idempotency on critical mutations
  Ensure submit/approve/export endpoints are idempotent against duplicate requests and network retries.

- Expand automated transition integrity tests
  Add scenario suites for multi-loop cases (clarification -> approval -> finance review -> export/recovery) to catch regressions early.

- Deploy canary/grayscale with hard rollback gates
  Use measurable SLO thresholds (error rate, queue aging, export fail rate, parse-fail rate) to auto-halt or rollback releases.

- Improve observability with end-to-end tracing
  Correlate case ID across frontend request, API transition, AI call, and queue jobs for faster incident triage.

© UMHackathon 2026